BIM Review

VIROO Studio Template

A quick and simple workflow to convert a BIM model into a collaborative VIROO XR Experience

Sergio Barrera Mayo
Virtualware CTO

Edward Martin
Chief Architect, Virtualware NA

CONTENTS

VIROO Studio Template – BIM Review 3
Specific Requirements 3
Template Project 4
Getting the Project Template 4
Loading the Template Project 4
Template Project Structure 5
How To Import Models 5
Repeating the Process from Scratch 7
Understanding the Transformation Workflow 10
What is a Ruleset? 12
Example Ruleset 13
Provided Rulesets in the Template 13
0 - Automatic Fixes Ruleset 14
1.1.1 - Remove Duplicated Metadata In Parents 15
1.1.2 - Fix Default Material 15
1.2.1 – Remove Cross Reference From ARK (Architecture) 16
1.2.2 - Remove Cross Reference from HVAC 16
1.2.3 - Remove Cross Reference from ELE (Electrical) 16
2.2.1 - Hide Sea Level Toposolids 17
2.1.2 - Hide Turning Free Space 18
2.1.2 – Hide Volume 18
3.1.1 - Fix Revit Floor Height Constraint 19
4.1.2 - Offset Floor AP2 20
4.1.3 - Offset up Floor VP1 20
4.1.4 - Offset up Floor VP2 20
5.1.1 - Materials Improve 21
5.1.2 - Prefabs Replacement 22
6.1.1 – Set Viroo Floors 23
How to Create New Rulesets 24
Working with Prefab Sustitutions 24
Avoid Future Dependencies on Imported Prefabs 24
Advanced Metadata Filtering 25
Viroo Studio Smart Metadata Filters 25
The Layer Visibility System 26
How it Works 26

Copyright 2025 Virtualware Company - All rights reserved.

How to Define Layers 26
Adding Viroo Teleport Points 28
Creating Teleport Points 28
Using Teleport Points in Runtime 29
The Teleport Points Inspector 30

Copyright 2025 Virtualware Company - All rights reserved.

VIROO STUDIO TEMPLATE – BIM REVIEW

In this document we cover the basic use of the ***BIM Review Template*** workflow, to process a BIM based 3D model coming from ***Autodesk Revit***, transform it inside Unity, making use of the **Asset Transformer Package** tools coming with ***Unity Industry license***, enhanced with some provided **VIROO Studio** for Unity extensions, in the form of Asset Transformer's actions. Although this example focuses on Revit models, the Unity Asset Transformer can also be used to import other model formats that carry metadata such as IFC.

SPECIFIC REQUIREMENTS

This template project makes extensive use of the Asset Transformer tools available through the ***Unity Industry*** license.

Asset Transformer enables the import of models from CAD and BIM tools and provides a way to create ***Rules*** that will be applied to imported assets, to automate the workflow to convert the CAD package model into an interactive VIROO project.

Copyright 2025 Virtualware Company - All rights reserved.

TEMPLATE PROJECT

GETTING THE PROJECT TEMPLATE

The first step is to get our provided template from the GitHub site. You can either clone our repository using a “git” client application to keep track of our changes and improvements or download a ZIP file directly.

Repository link:
https://github.com/virooxr/vst-bim-review.git

Compressed ZIP File:
https://github.com/virooxr/vst-bim-review/archive/refs/heads/main.zip

LOADING THE TEMPLATE PROJECT

This template uses an imported version of a sample 3D REVIT model which is free to use (under license terms covered by CC BY-SA 4.0). Original files were used during the importing process without any modification. This dataset is meant to be used for learning purposes only. Revit Files are not distributed in this template. If needed, the original Revit Files can be obtained directly from the source website: BIM4LCA BIM Files

BIM Files © 2024 by Nordic Innovation, an organization under the Nordic Council of Ministers.

Open the project template as any regular Unity Project (File > Open Scene > ... then navigate to the “Scenes” folder in your project) and then open the ***NordicLCA_Housing*** scene inside the ***Assets/Scenes*** folder. This project already has a lot of functionality out of the box, to make it compatible with ***Virtualware’s VIROO*** platform.

To set the editor camera into a proper position, do the following:

1. Inside the hierarchy, locate and select the object named ***Root → SceneConfig → SinglePlayerStart.***
2. Press ***[F key]*** to focus the camera on that object.

You should then see something like this.

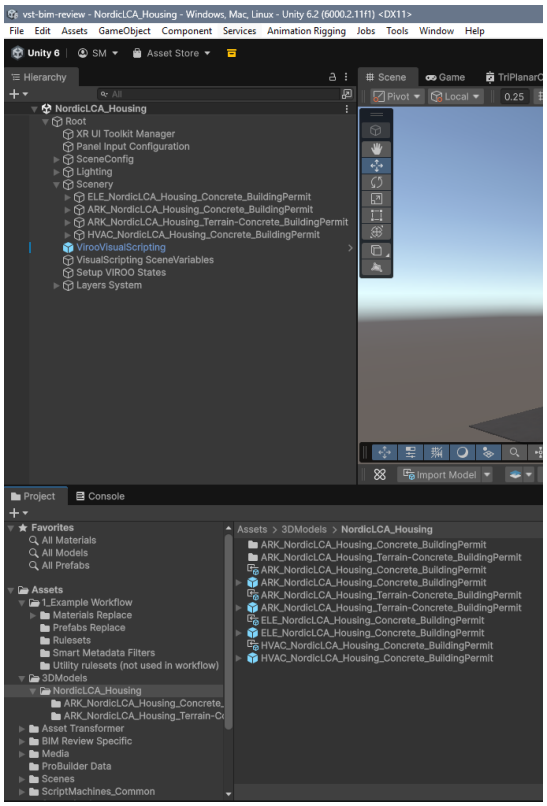
Copyright 2025 Virtualware Company - All rights reserved.

In this scene, there are several imported models inside the "***Scenery***" GameObject. Those are imported prefabs from Revit, each one consisting of a part of the architectural project.

This scene has already been processed by the Asset Transformer Custom workflow we have created for the project, that will be explained later.

TEMPLATE PROJECT STRUCTURE

The project has several important folders where the assets are stored:



- ***1_Example_Workflow***: Contains specific files which are part of the automation for this project.
    - ***Materials Replace***: Definition of materials to make automatic substitutions to improve appearance.
    - ***Prefabs Replace***: Contains definitions of some interactive objects like doors, to replace static ones coming from Revit.
    - ***Rulesets***: Metadata-based transformation workflows which will be applied automatically to the imported scene.
    - ***Smart Metadata Filters***: Definition of metadata-based filters with and / or logic, to be used to make complex selections.
    - ***Utility Rulesets***: Other rulesets not used in the automatic workflow.

- ***3DModels***: Imported prefab versions of the original Revit files: When importing a BIM or CAD model using ***Unity's Asset Transformer,*** you can decide whether to import them as scenes or as prefabs, in this case they were imported as prefabs.

HOW TO IMPORT MODELS

Now let's import our first Revit model. As a Unity Industry user, you have access to the full functionality provided by the Industry Package.

1. Click on the *Import Model* button in the *Asset Transformer* toolbar and browse to the location of the Revit file to import and select it. Asset Transformer will create a file in the folder *Assets/3DModels*. That file is not the imported file; it is the configuration to be applied to import the file.
2. In that configuration we need to be sure to have settings as follows:
    a. ***Import mode set to Prefab***
    b. ***Scale 0.001***. With this value, Revit models will be imported on the right scale.

Copyright 2025 Virtualware Company - All rights reserved.

   c. ***Import Metadata SET***
   d. ***Repair Instances UNSET***. Repair instances option causes some automatic fixes with some inconsistencies in the imported model which are not a good idea if we want to keep track of the metadata.
3. With that configuration set, click on the ***Import*** button in the settings inspector. The import process can take long, as it traverses the .rvt file. Once done, it will create another asset, with the same name but in the form of a ***Prefab*** containing the model.
4. That model can be then dragged and dropped as a child of the ***Scenery*** GameObject present in the scene.
5. Once in the scene, ***set its position to (0,0,0)***. If we use several Revit models overlapping, i.e.: having different layers, we need all of them to be imported at Zero, for the reference system to be common and consistent.
6. Once the prefab is instantiated in the scene, we need to perform an "***Unpack Prefab***". This way we will store transforms in the scene file itself, instead of relying on prefab, for these reasons:
   a. Some actions we will perform later cannot be automated in prefabs.
   b. With large geometries, the scene is quicker than referencing prefabs.
   c. We can use the Prefab as a backup to allow us rolling back to it while iterating creating Asset Transformer's Rules (explained later)

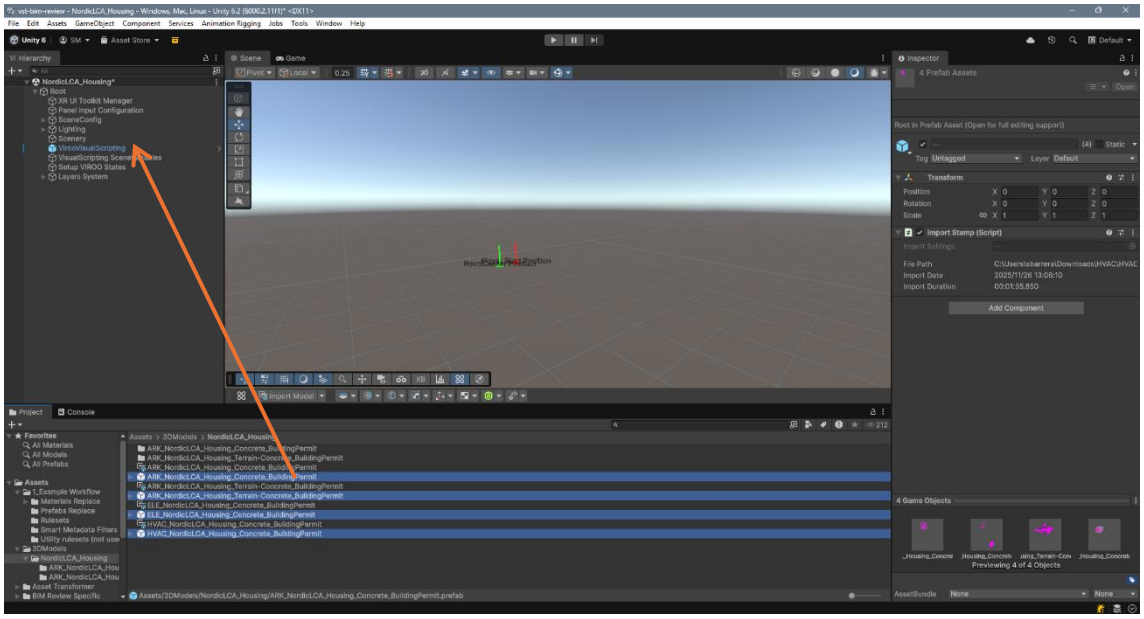
Important Note: Having models imported and "backed up" as prefabs can be very useful in the next steps, while we are experimenting and creating our workflow using transformation rules based on metadata, as we are going to be able to restore back the original prefab, and launch the transformation processes again.

Copyright 2025 Virtualware Company - All rights reserved.

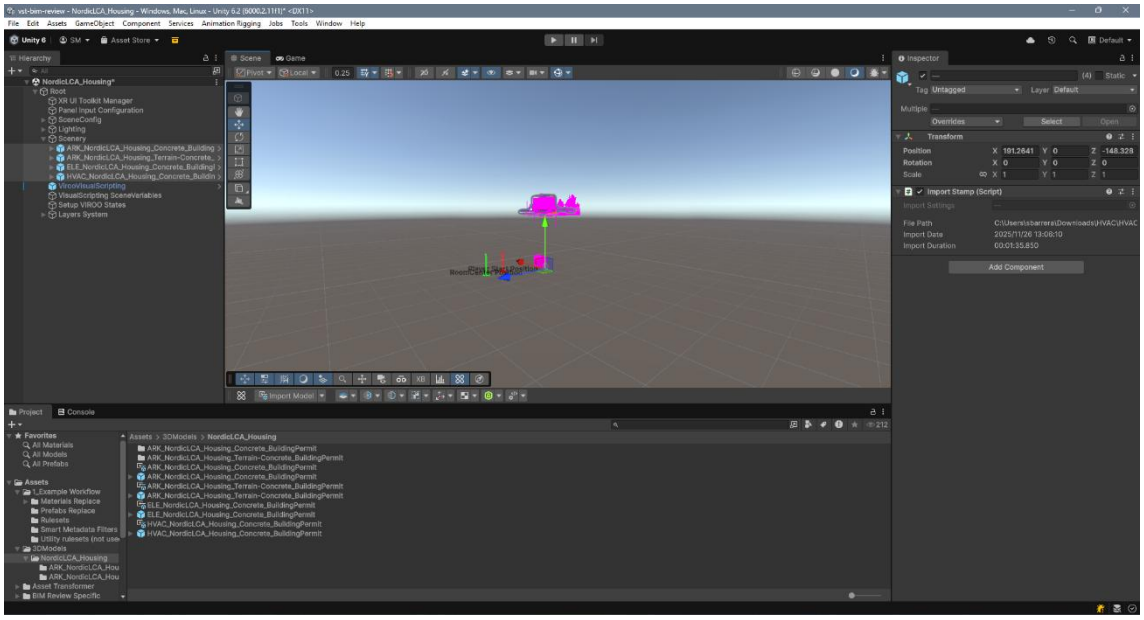
REPEATING THE PROCESS FROM SCRATCH

Let's repeat the ***Automatic Workflow*** ourselves, to understand how this works, and how it can save us a lot of time while working with metadata.

1. In the Hierarchy window, remove the architectural models imported from the Scene:
    a. Select every GameObject in the hierarchy below "***Scenery***".
    b. Remove them using "Delete/Remove key"

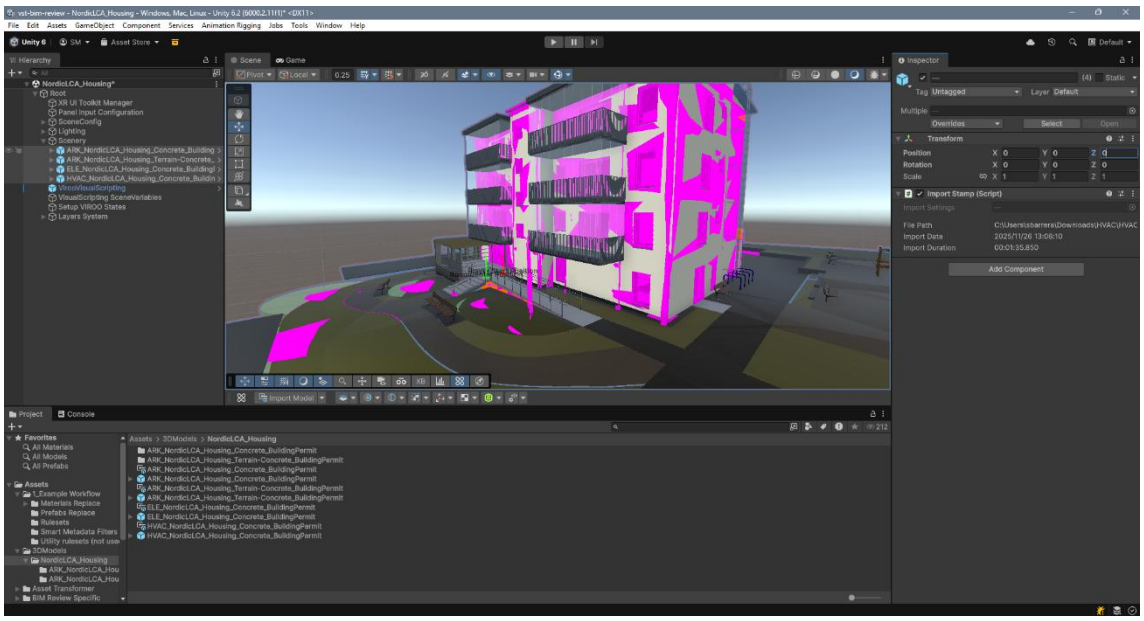


2. From the Project, drag and drop prefabs again inside ***Scenery***
    a. Prefabs are marked in blue in the previous picture. You can make a multiple selection by keeping [***Ctrl key***] pressed.
    b. Then drag them over the Scenery GameObject to make them children of it. As it is a large model, this can take some time.

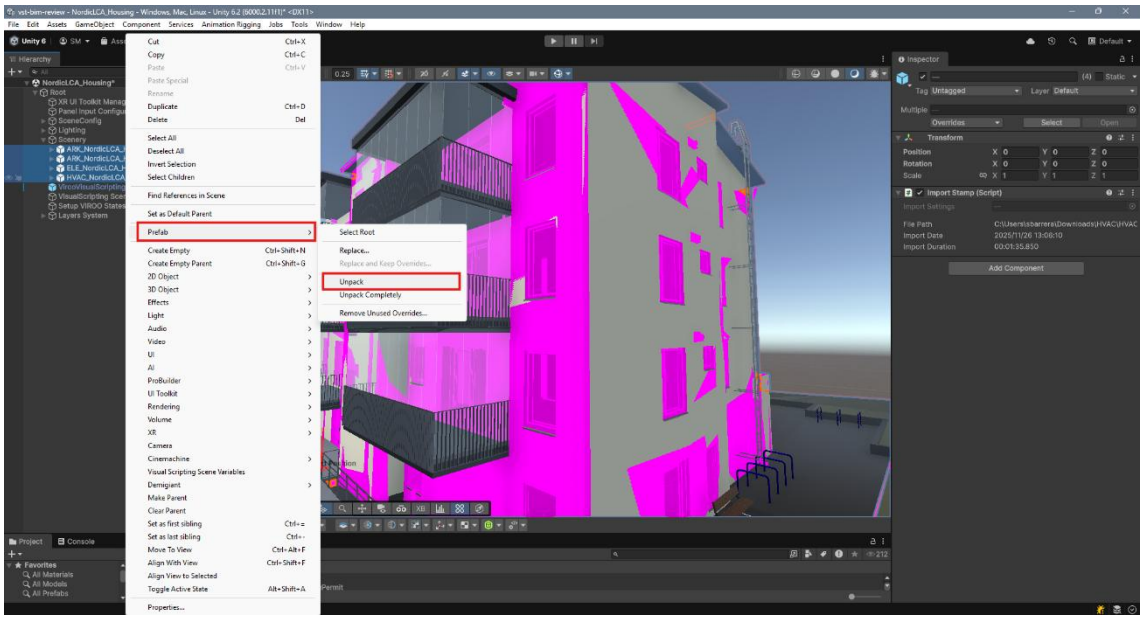


Copyright 2025 Virtualware Company - All rights reserved.

3. Now we prepare the models for the ***Automatic Workflow*** in these two steps
    a. **Set the position of every imported model to the origin**: Once you see the models in the Scene, for the models to be coordinated, set their locations in the ***Transform*** component to the same value ***(0,0,0)***. You can select each one and set its location to ***(0,0,0)*** separately or all together. (multiple selection works with hierarchy objects as well).



    b. **Unpack prefabs**: The Revit models have been imported using the "***prefab***" import workflow of the Unity Asset Transformer, to have a "***backup***" version of the models without modifications, but for the Rulesets to work, we need their objects to be part of the scene, instead of external references. Having selected them in the hierarchy, right click and then do ***Prefab → Unpack***



Copyright 2025 Virtualware Company - All rights reserved.

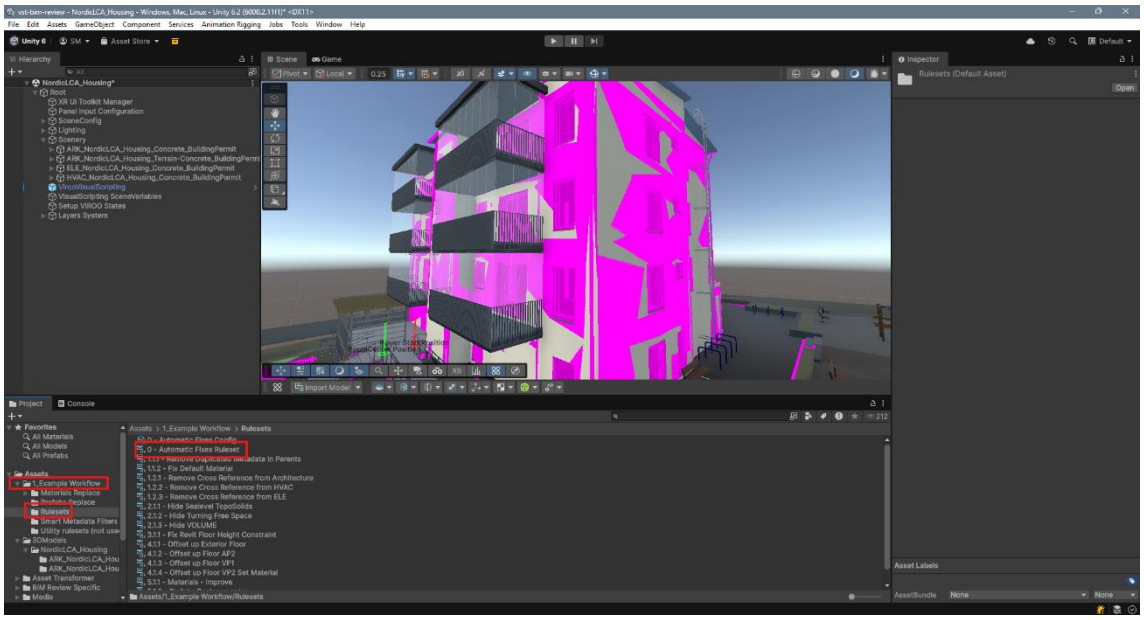
c. After the process is completed, the color of selected objects in the hierarchy panel will change from blue to white, as only prefabs are shown in blue. This indicates that objects are no longer prefabs.

Optional: Now this could be a good time to check the model, so we could notice which kinds of problems needed to be solved. Each one of these problems observed will be solved using different ***Rulesets.***

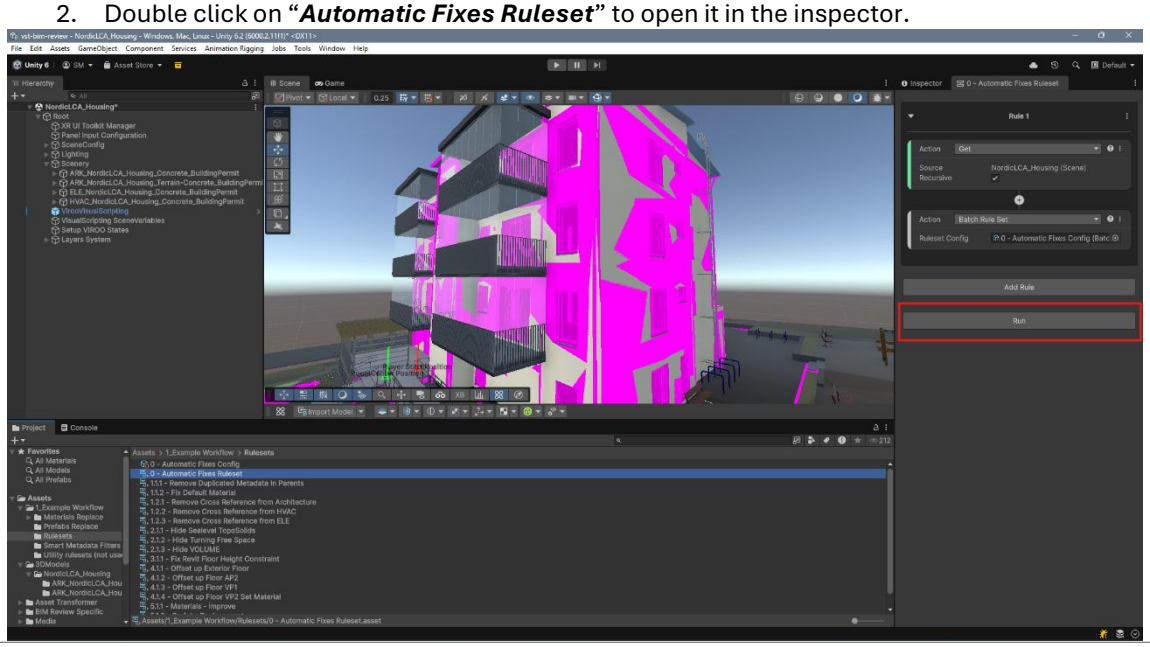
In the next step we are running the automatic workflow which is going to make a lot of automatic adjustments. To see specific details about those automatic improvements, we invite you to continue reading the next chapters of this document.

Let's launch the process,

1. in the project panel, locate the file "***Automatic Fixes Ruleset***", inside the path ***Assets/1_Example Workflow/Rulesets***

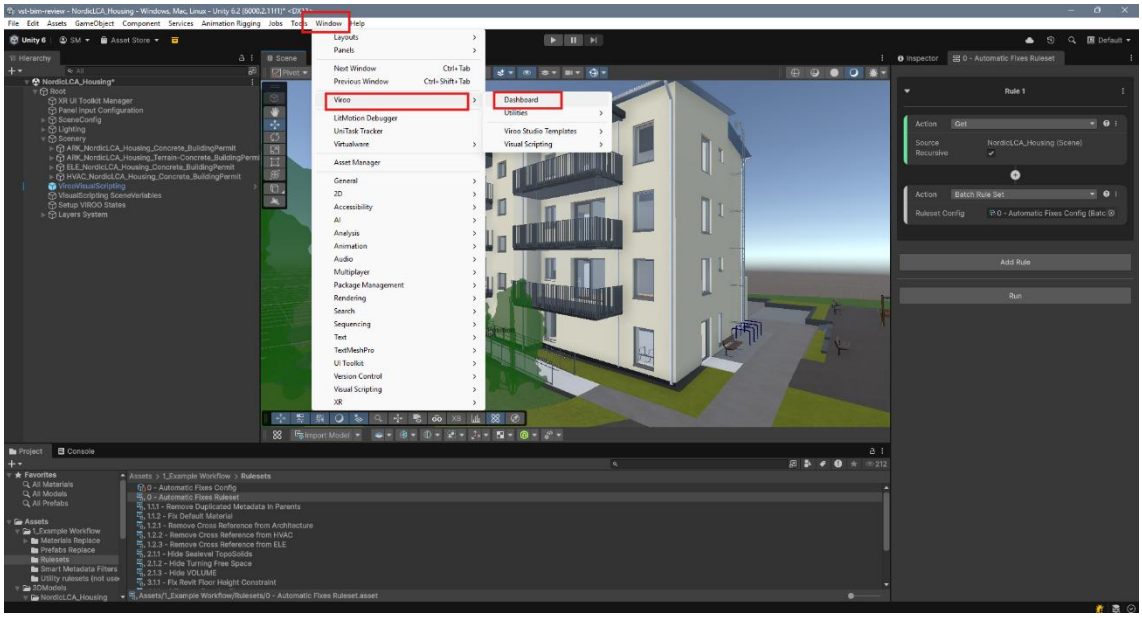


2. Double click on "***Automatic Fixes Ruleset***" to open it in the inspector.



Copyright 2025 Virtualware Company - All rights reserved.

3. To start the process, now click on the [***Run***] button.
    a. This action will take some time.
    b. It will process the whole scene, iterating over a batch of configured ***Rulesets*** and making the configured changes in each one of the ***Rulesets***
4. When it finishes, we only need to take an additional step to enjoy our interactive VIROO experience.
    a. One of the rules has created a bunch of interactive and network synced "***Doors***" which will allow us to open them using VR controllers or the mouse.
    b. Those objects are instances of prefabs, but VIROO needs unique "***network ids***", so let's fix.
    c. We can solve that issue in the VIROO dashboard



    d. In the ***VIROO dashboard***, go to the tab ***Project Validation***
    e. Click on ***Fix All.***

Now if you hit play in the Unity Editor, you can enjoy an interactive experience.

UNDERSTANDING THE TRANSFORMATION WORKFLOW

The template has a custom transformation workflow, processing the imported model to improve its appearance, and making it VIROO interactable.

This is done making use of "Rulesets" in the Asset Transformer, which are going to rely on the imported metadata to script the automation. That will allow things like:

- To improve the imported model's general appearance.
    - Making automated substitutions of materials.
    - Creating offsets to avoid geometry overlaps.
    - Remove z-fighting.

- To solve importing issues related to Revit reference systems
    - Modify transformations of objects depending on Revit constraints, i.e.: ceiling compounds).
    - Replace/ remove mirrored object instances
    - Normalization of scaling.

Copyright 2025 Virtualware Company - All rights reserved.

- Optimizing models with high polygon count
    - Decimating models to a target.
    - Creating automatic levels of detail.

- Make some of the items interactable through the VIROO systems.
    - Making doors animated and interactable responding to mouse and controllers.
    - Making selected furniture to be grabbable with the VR controllers.

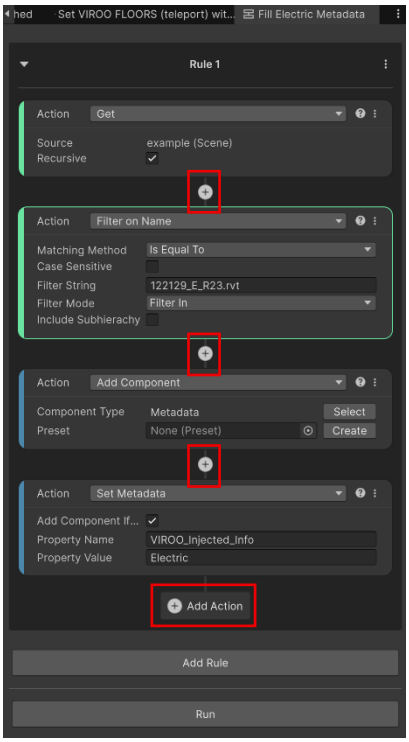
These “Rulesets” will be persistent in the project, so the automation can be reused in the future.

The idea behind this is to have the ability to run an “automated process” each time we need to re-import the models as they evolve, not have to edit everything manually each time we import, but just adding new features.

Furthermore, if our Revit different projects follow a homogeneous workflow, it is possible to define a common workflow based on persistent Rulesets, allowing them to be reused across projects.

Copyright 2025 Virtualware Company - All rights reserved.

WHAT IS A RULESET?



An "Asset Transformer's Ruleset" is a way to configure a workflow to be applied to a set of GameObjects in context of the Unity Editor.

Rulesets are defined as a sequence of ***Asset Transformer Actions.*** Each one of these actions will perform a task with the current selection of GameObjects and then will pass the resulting GameObjects to the next one in the sequence.

There are specific actions which allow us to select objects, those are the ***Filter Actions.***

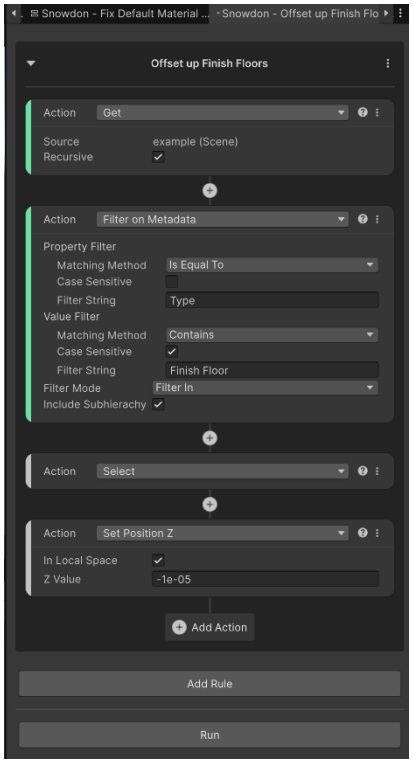
Unity provides a set of *Asset Transformer Actions,* which can be added in the sequence clicking on ***[+]*** or ***[+ Add Action]*** buttons and VIROO platform extends that list with a new set to make things easier. That can be found in the regular Actions list, inside the specific ***VirooStudio*** Category.

A Ruleset can be triggered over the current selection of objects by clicking on the ***[Run]*** Button.

Later we will see as we also have a convenient way to run a ***Batch of Rulesets***. This is basically a way to create sequences of Rulesets that will be executed in order, to potentially have a full automation triggered after importing with just one click.

Copyright 2025 Virtualware Company - All rights reserved.

EXAMPLE RULESET



As stated***, rulesets*** are composed of ***actions*** which are executed from top to bottom. Each ***action*** can get an ***input GameObject list*** from the previous output (or from the current selection in the Unity Editor for the first one), make its work and can potentially generate another ***output GameObject list*** for the next ***action.***

This example Ruleset acts upon every "Finished Floor" object and moves it upward very slightly to avoid overlap with other "structural" objects. This prevents visual artifacts which can result when multiple geometry objects exist in the same plane.

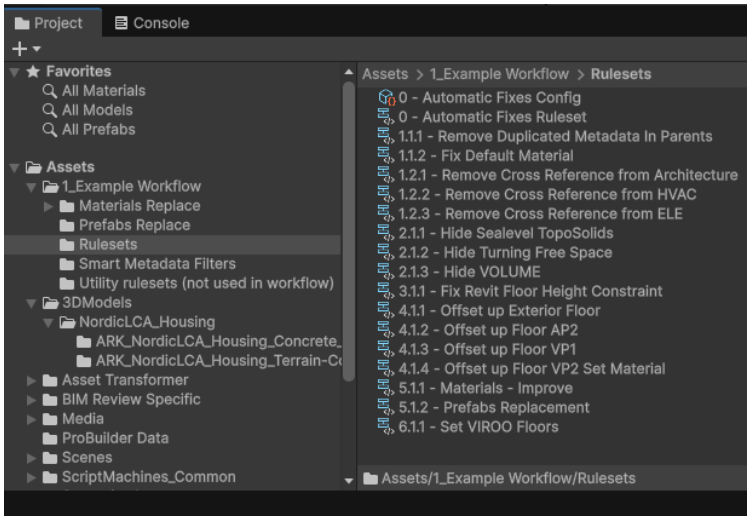
1. First: "***Get***" Action retrieves every GameObject in the scene and passes them to the next stage. Normally this action will be the first one in any workflow.
2. "***Filter on Metadata***": For any of the objects in the input, checks if its metadata fields match with the provided configuration. In this case the objects need to have a "***Type***" property containing the text "***Finish Floor***", if they match then include them in the output for the next stage.
3. "***Select***": Select the GameObjects (It does nothing but select objects, used to debug our rules and to can be used to make complex selections in the Editor).
4. "***Set Position Z***": For those objects reaching this stage, its transform.localPosition.z is set to the value -0.00001.

PROVIDED RULESETS IN THE TEMPLATE

Review of the existing rulesets to understand how they work.

This template already includes a set of rulesets which are quite convenient to use in any Revit project, but you can extend or change existing ones to fulfill your project's specific needs. These rulesets can be found in the ***"Assets/1_Example Workflow/Rulesets".***

The rulesets have been named with numeric prefixes, allowing them to be grouped in chapters.



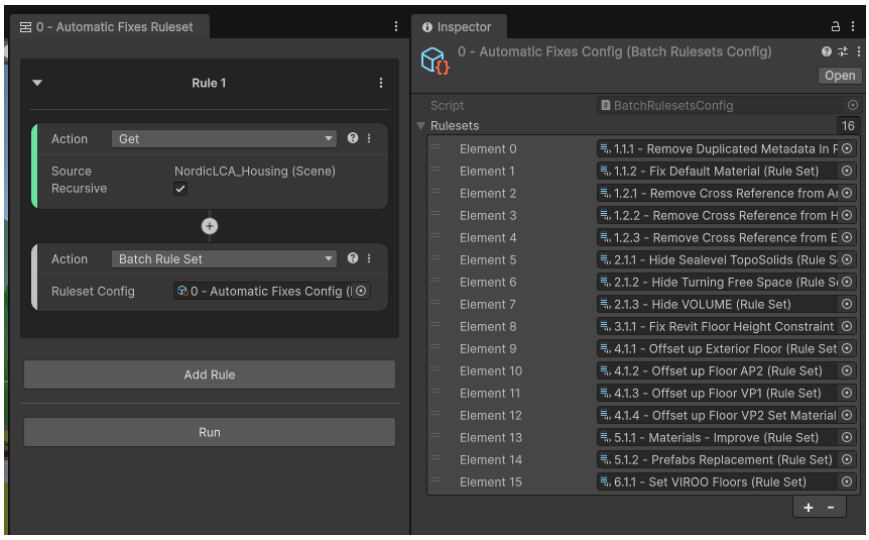
This is good practice, as some of the Ruleset execution results could depend on a previous step, so we need to be careful about the ordering.

In the next sections we will be covering the purpose of each ***ruleset***, and how they work.

Copyright 2025 Virtualware Company - All rights reserved.

0 - AUTOMATIC FIXES RULESET

This ruleset is an aggregate of the other rules that will be used to run the full automated process of fixing our Architectural model in just one click.



It iterates over the set of rules configured from top to bottom over the scene (or the current selection done, depending on the selected context). In some cases, the order of operations is important.

The first Action (Get) makes the selection, and the second Action contains a reference to a configuration Asset (Batch Ruleset Config), where the list of Rulesets to be executed is stored in order, from top to bottom. This list can be edited using standard Unity ways, drag and drop, adding new elements, and removing them. Elements can be dragged up or down, to change order as well.

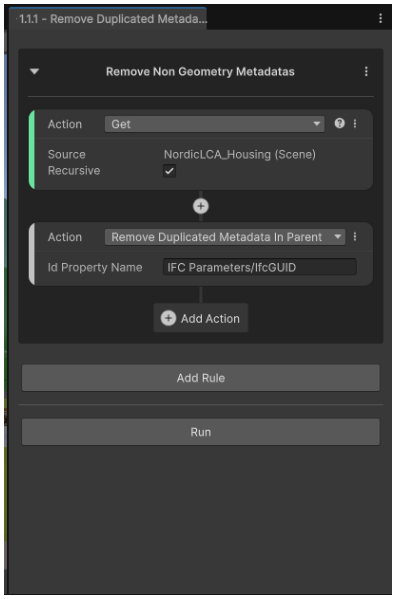
This list can be extended with more rulesets:

1. Create the new ruleset as desired.
2. Click on the small “+” button to create a placeholder for the new ruleset
3. Then drag & drop the ruleset asset just created.

This way, the changes applied to a project can evolve depending on new needs.

Copyright 2025 Virtualware Company - All rights reserved.

1.1.1 - REMOVE DUPLICATED METADATA IN PARENTS



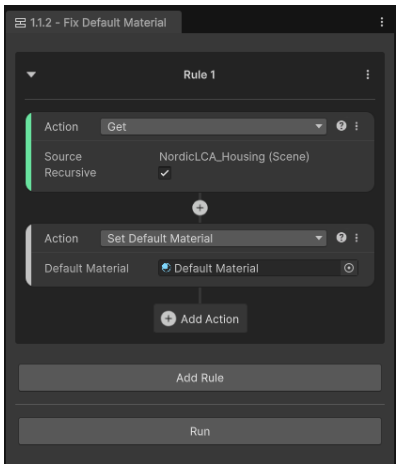
When importing models coming from Revit, metadata in parent objects in the hierarchy is sometimes duplicated from the first child having metadata.

In our workflow this is not convenient, as we will be replacing some objects with “prefabs” automatically, and we will rely on metadata. For that reason, we need to clean-up those metadata objects which are replicated in parents.

To solve this, we created a specific Asset Transformer Action in the ***VirooStudio*** category, which looks for the same value of a specific field in the parents of those objects having metadata, and if found, it removes them.

The Action Remove ***Duplicated Metadata in Parent*** action allows us to configure the property field we want to use to compare unique instances of objects, in this case we are using ***IFC Parameters/ifcGUID*** to differentiate unique object instances coming from Revit.

1.1.2 - FIX DEFAULT MATERIAL



This is a very simple ruleset that looks for missing or null materials in the scene and change them with a “Default” that can be set as a parameter.

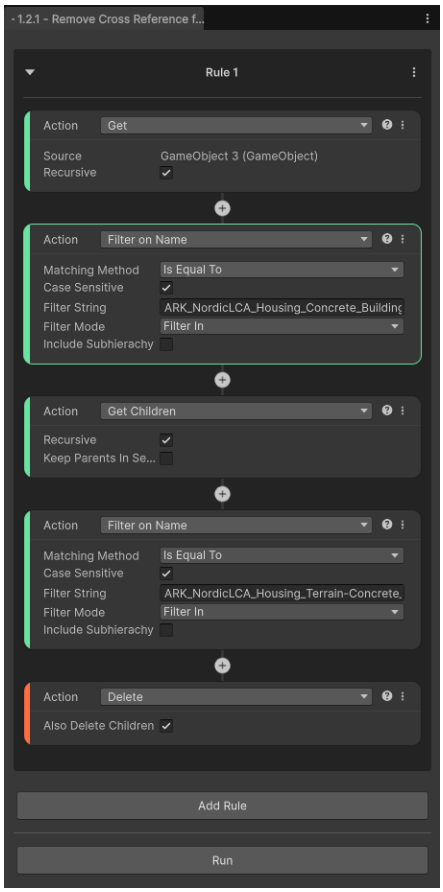
In this case, to ease the use, it is not necessary to filter objects, because the ***VirooStudio/Set Default Material*** action looks for missing materials internally.

Copyright 2025 Virtualware Company - All rights reserved.

1.2.1 – REMOVE CROSS REFERENCE FROM ARK (ARCHITECTURE)

1.2.2 - REMOVE CROSS REFERENCE FROM HVAC

1.2.3 - REMOVE CROSS REFERENCE FROM ELE (ELECTRICAL)



A Revit file can often contain references to other RVT project files. For example, an architectural model can have references to the structural model.

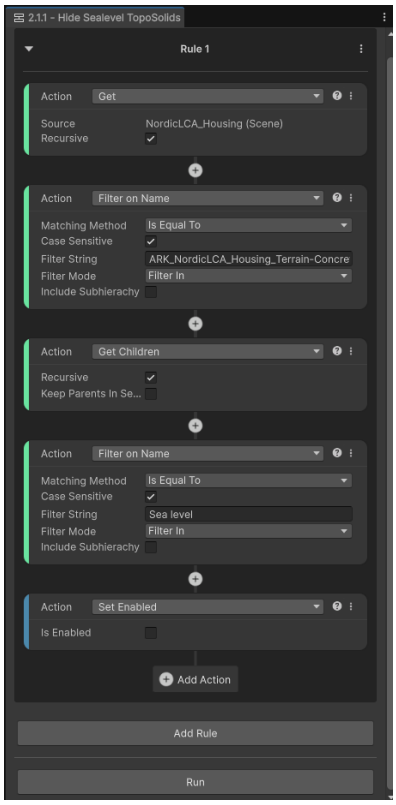
As we are mixing several files, to visualize them synchronized, some of the references coming from cross references could interfere, or be at wrong locations, as their complete structure is not present.

In these cases, it is a good idea to remove cross references and rely only on objects we know each of the model "RVT file" provides.

We filter a specific name of GameObject (***Filter on Name***), grab their children recursively (***Get Children***), look for the one we want to remove (***Filter on Name*** again) and finally ***Delete*** it.

Copyright 2025 Virtualware Company - All rights reserved.

2.2.1 - HIDE SEA LEVEL TOPOSOLIDS



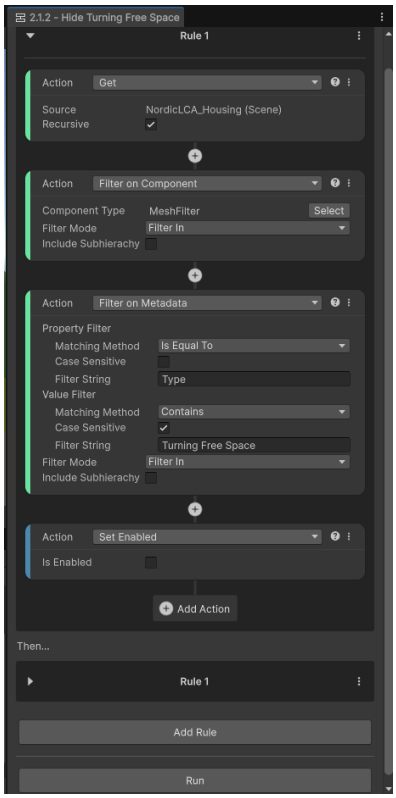
This Ruleset allows us to hide objects in the hierarchy by name.

It is built in a similar way to the previous one. It selects a parent object by name, then grabs its children recursively and looks for the one to hide.

Instead of removing the object found, the ***Set Enabled*** action disables the GameObject, but keeps it in the hierarchy, so it could be potentially enabled again if needed.

Copyright 2025 Virtualware Company - All rights reserved.

2.1.2 - HIDE TURNING FREE SPACE



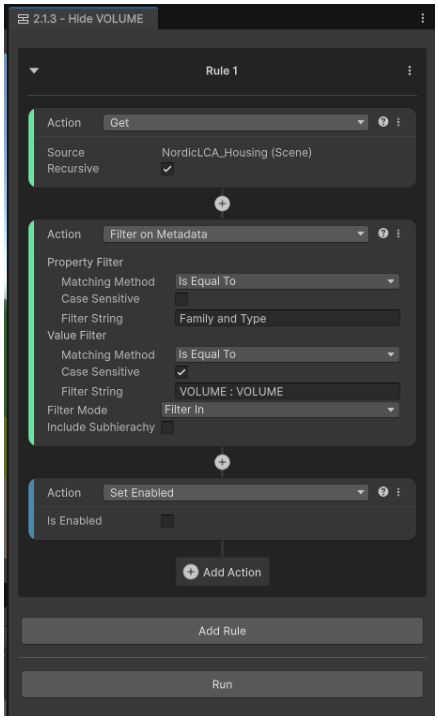
In the model, there are several auxiliary objects representing some volumes defined to be compliant with accessibility. These volumes are logical.

We can create rulesets using filtering of some fields present in the metadata with a specific value, and then using the ***Set Enabled*** action, we can disable their GameObjects, to not have a visible presence in the scene.

This ruleset has several rules chained, as there are objects with different “Types” we need to set hidden.

Group rules inside a ruleset are a good idea if we need to make the same process but using different property values in metadata.

2.1.2 – HIDE VOLUME



Here we look for the objects for which “Family and Type” property value is equal to “VOLUME : VOLUME”.

And then disable them.

Copyright 2025 Virtualware Company - All rights reserved.

3.1.1 - FIX REVIT FLOOR HEIGHT CONSTRAINT

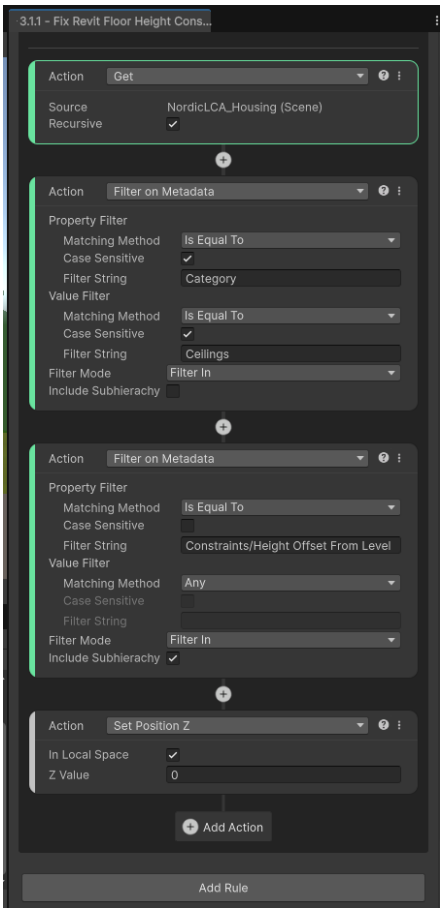
Some objects coming from Revit can potentially be placed in the wrong vertical axis coordinates.

That is due to the wrong interpretation of some parameters when the objects are placed depending on others and use the property "***Constraints/Height Offset From Level***".

This problem is usually present in some specific systems like "Compound Ceilings", being placed with a "z" offset instead of in their proper location.

These Ruleset solves that problem using the sequence:

1. Get objects present in the scene.
2. Filter on Metadata for objects in "***Category***" with value "***Ceiling***"
3. And filtering again those having the property "***Constraints/Height Offset From Level***" defined with any value.
4. Set its transform ***localPosition.z*** to 0.

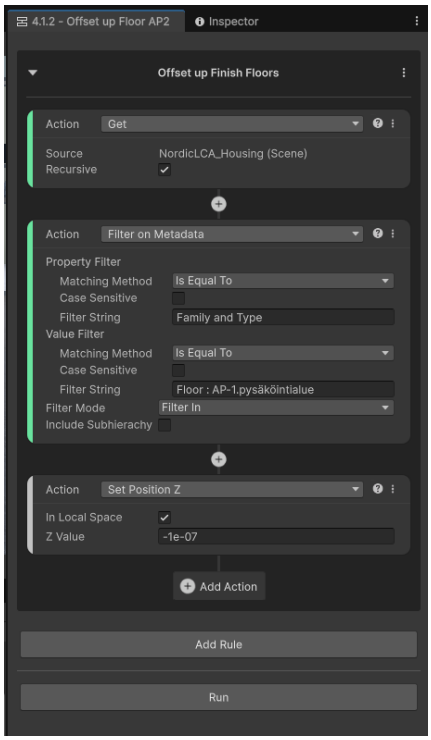


Copyright 2025 Virtualware Company - All rights reserved.

4.1.2 - OFFSET FLOOR AP2

4.1.3 - OFFSET UP FLOOR VP1

4.1.4 - OFFSET UP FLOOR VP2



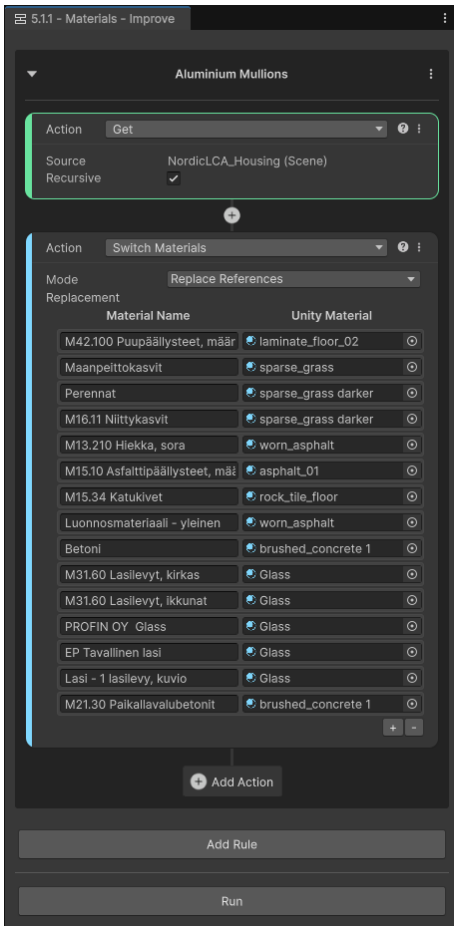
All these rulesets look for specific objects in the scene, using a single Property Name and a Value, and create offsets in the transformations, to avoid overlapping visually with other objects (z-fighting).

Filter on Metadata is used to select objects to be offset.

Set Postion Z with a very small value is used to move up the objects (notice that the value is negative and local space is used, as imported objects' local transformation coming from Revit vertical axis is -z)

Copyright 2025 Virtualware Company - All rights reserved.

5.1.1 - MATERIALS IMPROVE



This ruleset is used to switch some of the materials coming from Revit with others defined in a project's folder to make the scene nicer and more realistic.

As the materials are found by "***Material Name***", and the replacement materials are defined in a persistent way in the project not to depend on the imported prefabs, next time we re-import the project, the references to them will continue to work.

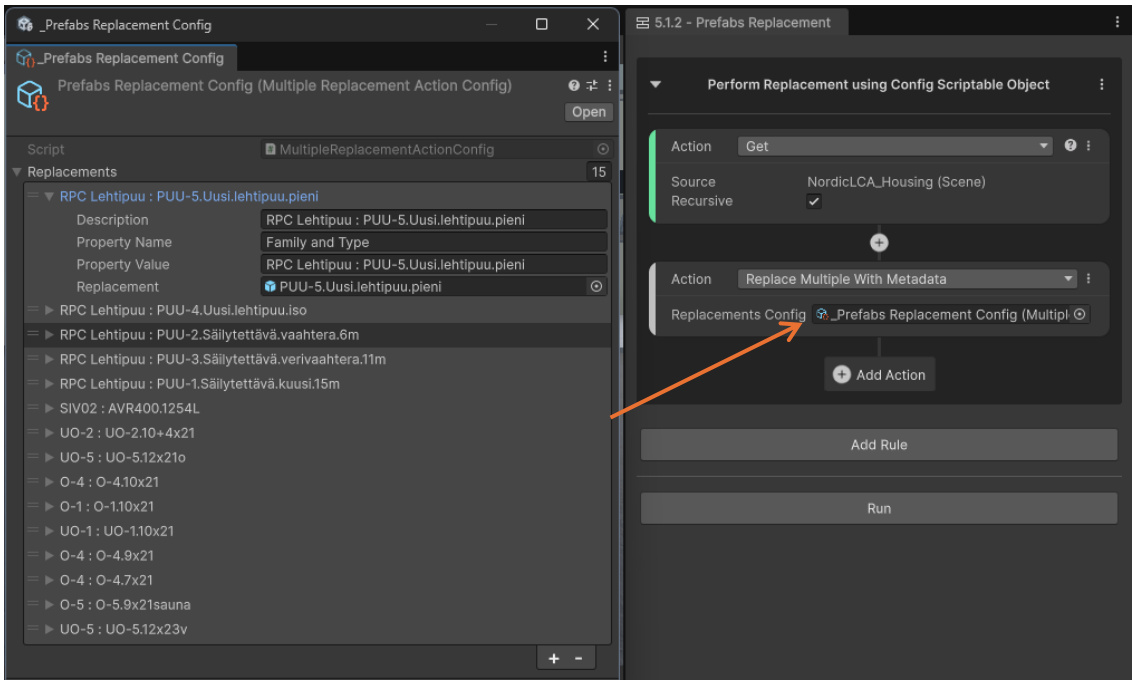
Some of the defined Materials are using a ***Triplanar Shader***, which projects realistic Diffuse, Normal, and ARM textures using local space Triplanar mapping provided by the Viroo Studio Template.

This is very useful as normally geometries coming from Revit do not provide texture maps or UV coordinates, with this approach the texture mapping is generated in the GPU.

Copyright 2025 Virtualware Company - All rights reserved.

5.1.2 - PREFABS REPLACEMENT

This Ruleset provides a way to configure automatic substitutions of every instance in the scene matching with any of the configurations provided in the ***Replacements Config*** asset.



The ***Replacement Config*** asset consists of a list of entries where each entry is a particular definition of the criteria to match a kind of object based on a metadata field, and the prefab we want to use as replacement when the criteria match.

For this purpose, we normally use "Type and Family" as property fields when working with Revit models, as it identifies a class of instance in Revit, but other fields can be used as needed.

This works as follows:

1. Get every object in the Scene
2. Run the ***Replace Multiple With Metadata*** Asset Transformer Action provided inside ***VirooStudio*** category.
    a. Iterate over all objects and for each of the objects, compare metadata with configurations, and perform replacements for any matches.

For example, the first entry states: If ***"Family and Type"*** is equals to ***"RPC Lehtipuu : PUU-5.Uusi.lehtipuu.pieni' Canopy"*** then replace it with the supplied prefab ***"PUU-5.Uusi.lehtipuu.pieni".***

As the project evolves, you can create new prefabs and add them to the list, so next time you re-import, you won't need to redefine everything from scratch, as these configurations are stored as a ***Scriptable Object*** in the ***"Assets/1_Example Workflow/Prefabs Replace"*** folder.

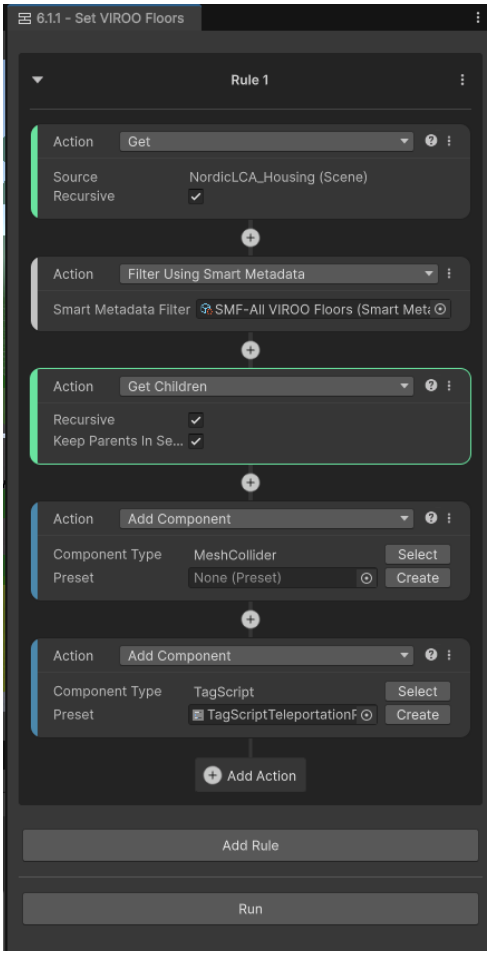
Pro Tip 1: When you create prefabs based on imported objects, try to leave "***Metadata***" components in the prefab untouched, this way you will always have a way to know its source and for example be able to replace it again.

Pro Tip 2: More than one ***Replacement Config*** asset can be created to have several sets of prefabs with different results in the same project. Example: replace prefabs without geometry but with

Copyright 2025 Virtualware Company - All rights reserved.

metadata, to keep objects tracked but without visual presence: ***Unity Create Menu → Viroo Studio → Templates → Multiple Replacement Action Config***

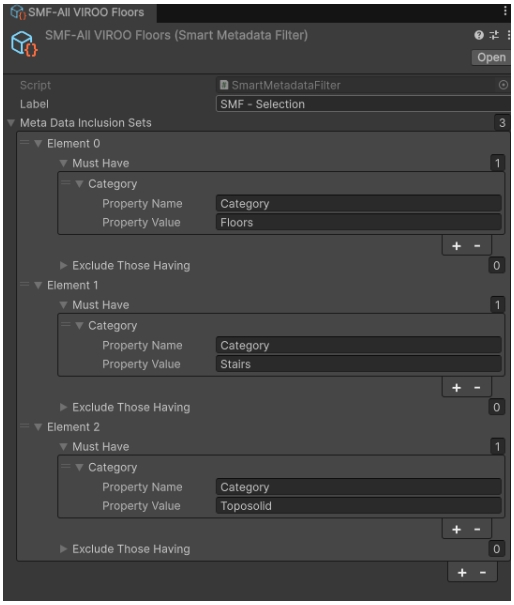
6.1.1 – SET VIROO FLOORS



This Ruleset allows us to select objects using a ***Smart Metadata Filter*** to add needed components to objects, for them to be walkable floors in VIROO.

The ***Smart Metadata Filter*** provided by VIROO consists of a persistent configuration done in other Assets, that can be referenced, to allow us to make complex selections of objects, using ADD / OR / NOT logic.

In this example, we look for objects using this configuration:



It will filter in every object using the Metadata property name "***Category***" with values: "***Floors***", "***Stairs***" or "***Toposolid***".

And then for those objects found, it will add a ***MeshCollider*** component, to allow Unity to detect collisions, and a ***TagScript*** with a specific preset we use in the template to let VIROO know these are walkable floors.

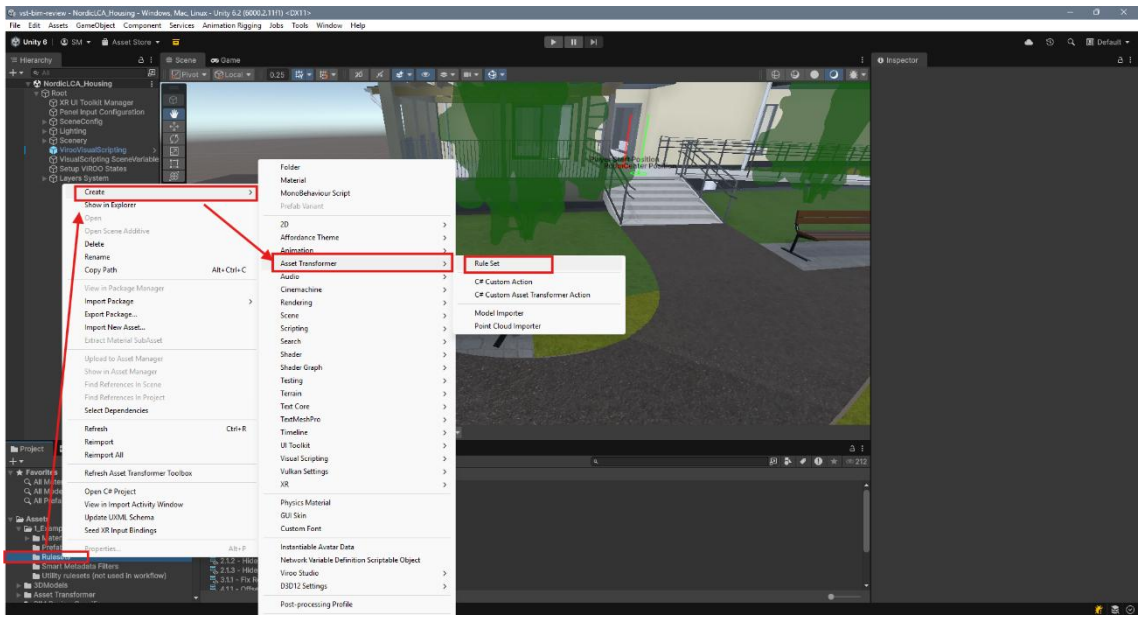
If you need to modify or add more "walkable" VIROO floors, you will only need to extend the ***Smart Metadata Filter*** to include your objects.

Copyright 2025 Virtualware Company - All rights reserved.

HOW TO CREATE NEW RULESETS

Creating new rulesets is done using the standard “Create” workflow in Unity:

1. ***Right Mouse Button*** over the target ***folder*** in the ***Project panel.***
2. Click on ***Create.***
3. Then click on ***Asset Transformer***
4. Then click on ***Ruleset.***



This will create a default Ruleset with a Rule having the default Action to select objects from the Selection context (current selection or Scene if nothing selected). Finally, set an appropriate descriptive name to the ***Ruleset*** asset file.

Advanced Tip: Once the Ruleset is properly configured to fit our needs, it can additionally be added to the “***Automatic Fixes Ruleset***”, to be part of the automated process.

A Ruleset asset can also be duplicated, as any other Unity asset by selecting it in the ***Project Panel***, and then pressing ***[Ctrl+D]*** shortcut key.

WORKING WITH PREFAB SUSTITUTIONS

AVOID FUTURE DEPENDENCIES ON IMPORTED PREFABS

When creating prefabs to be used by the ***“Prefab Replacement” Ruleset***, you need to be sure that any reference Asset the prefab uses, like for example materials, or meshes, are not part of the imported Revit prefabs, not to lose references when you potentially reimport un updated version of the model.

If you need to reference some 3D mesh, material, or whatever asset inside the prefab, you can ***duplicate*** that specific asset in the project, to have an independent version of it.

When you duplicate an asset inside a prefab, Unity creates a persistent version of it not dependent on the source asset, so could potentially remove or replace the original prefabs and the new asset will continue to work.

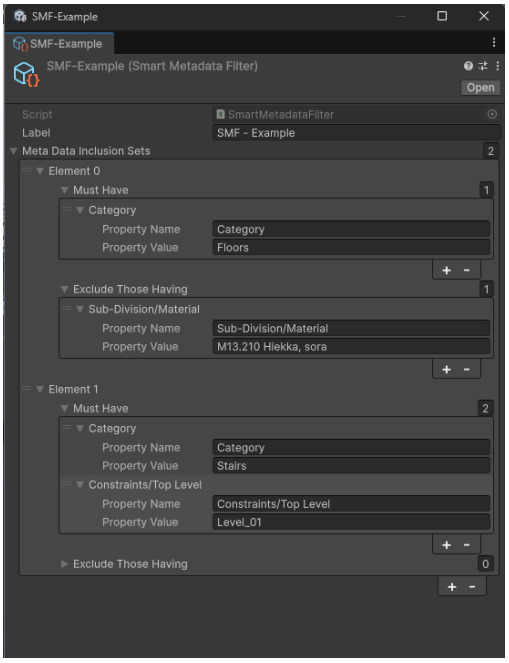
Copyright 2025 Virtualware Company - All rights reserved.

ADVANCED METADATA FILTERING

Unity provides an ***Asset Transformer*** action named ***Filter on Metadata*** which can be used in almost every case, but if you need to apply complex AND / OR / NOT logic to select objects for transformation, you can make use of the Smart Metadata Filter action provided inside ***VirooStudio*** category.

VIROO STUDIO SMART METADATA FILTERS

Smart Metadata Filters are a way to create AND / OR / NOT logic depending on metadata, that can be evaluated even in runtime.



Smart Metadata Filters are composed of a list of ***Metadata Inclusion Sets:*** If an object meets conditions specified in any of these sets, it will be considered as included. So, they are useful to create "***OR***" additions. Those objects in ***Element 0*** -OR- those Objects in ***Element 1,*** and so on.

A Metadata Inclusion Set has two internal sets of Metadata fields:

1. ***Must Have***: Determine fields an object needs to have with its specific values. This enables constructing "***AND***" logic.

2. ***Exclude Those Having***: Determine object exclusion depending on defined metadata property fields and values. This enables "***NOT***" logic.

In this example we have two inclusion sets:

The first one includes objects having a property field name ***"Category"*** whose value is ***"Floors"***, but excluding (***NOT***) those having the property name ***"Sub-Division/Material"*** defined as ***"M13.210 Hiekka, sora"***.

The second includes objects in the meeting both conditions: ***"Category"*** with value ***"Stairs" AND "Constraints/Top Level"*** with value ***"Level_01"***

As seen, this is very powerful, allowing us to create complex logic, in a human understandable way:

The logical algebraic notation could be represented as:

(["Category"]=["Floors"] where **NOT** ["Sub-Division/Material"]=["M13.210 Hiekka, sora"])
OR
((["Category"]=["Stairs"]) **AND** (["Constraints/Top Level"]=["Level_01"]))

Copyright 2025 Virtualware Company - All rights reserved.

THE LAYER VISIBILITY SYSTEM

The template provides a standardized way to create dynamic layers, based on groups of objects that will be presented in a layer menu, to enable the end user to activate or deactivate visibility.

HOW IT WORKS

The layer systems benefit from ***Smart Metadata Filters***. A Smart Metadata Filter determines a set of objects which are grouped into a layer, and it is executed in the startup of the scene, so it will always be updated, even if you import new objects to the scene.

The layers are already prepared to synchronize with other users in the VIROO session, so you do not have to care about the complex synchronization process.

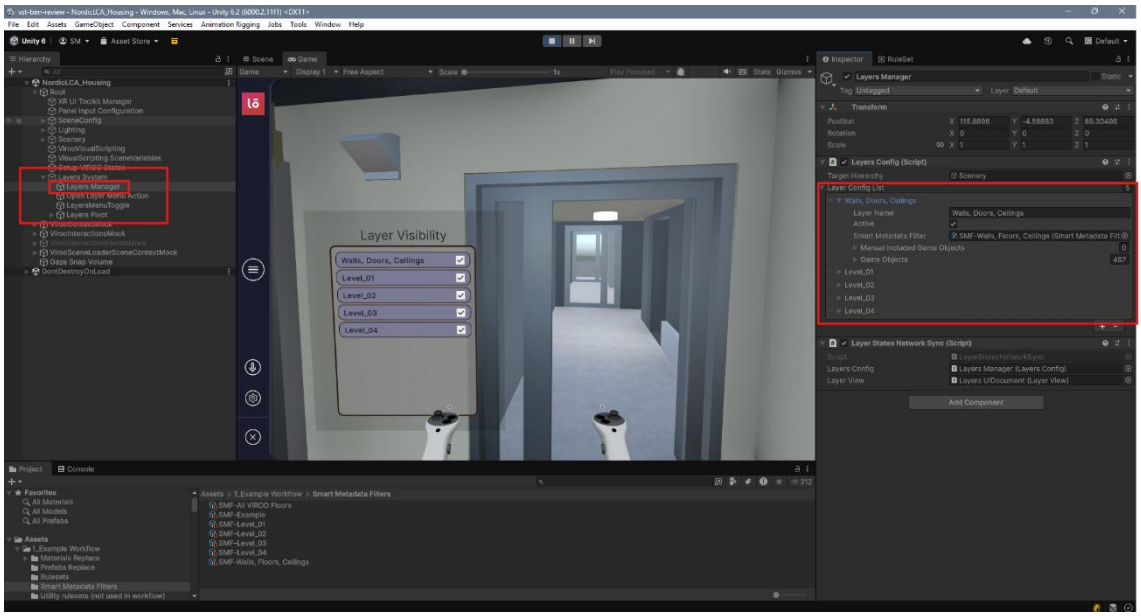
HOW TO DEFINE LAYERS

To define layers, we have included a "***Layers System***" object in the Scene, which has inside the full set of visuals, and programming logic to work, and relate with "***Metadata***" coming from Revit, using ***Smart Metadata Filters.***

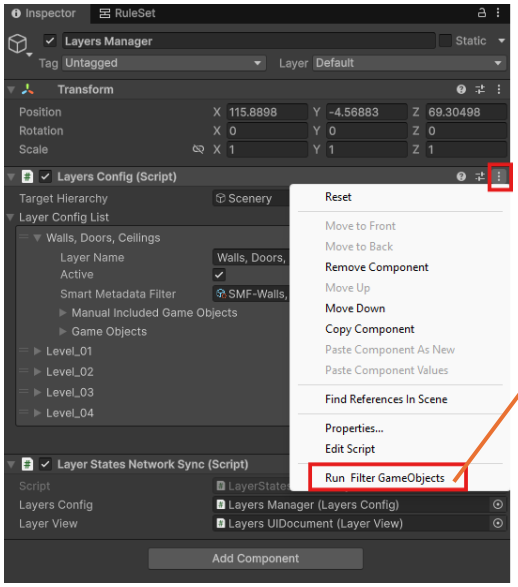
A layer consists of a name, to be shown in the menu, and a ***Smart Metadata Filter*** definition to determine which objects to include.

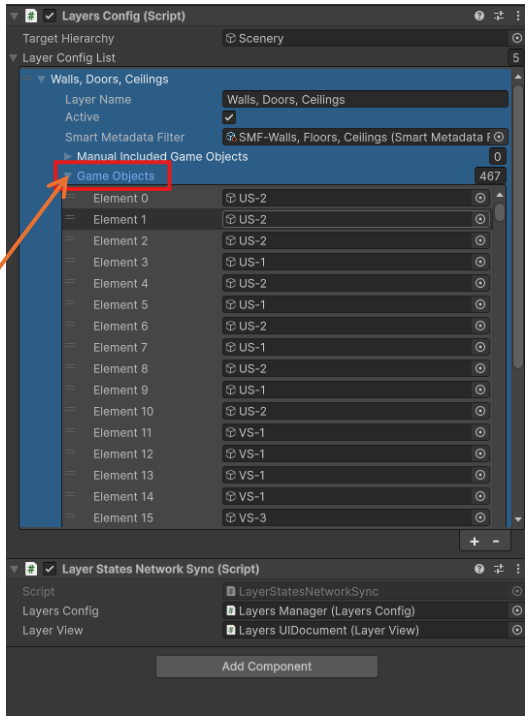
Additionally, it has a list of "***Manually included GameObjects***", which provides a way to drag and drop custom objects from the hierarchy if needed.

Copyright 2025 Virtualware Company - All rights reserved.



Advanced Tip: To check if the object selections are correct, using the ***Layers Config***'s component context menu, it is possible to "preview" the lists of GameObjects which will be selected. Go to the ***three-dot*** context menu in the right top of the component, and then click on ***Run Filter GameObjects***, it will perform the selection and fill in the list. This step is ***optional***, as it will be automatically performed when playing the content.





Copyright 2025 Virtualware Company - All rights reserved.

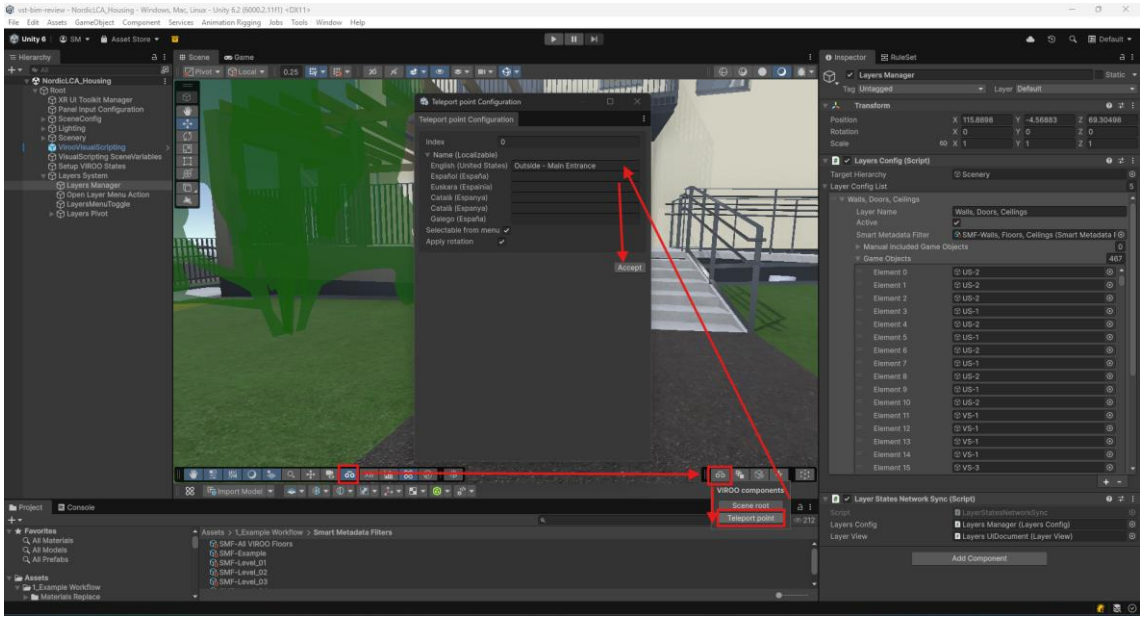
ADDING VIROO TELEPORT POINTS

CREATING TELEPORT POINTS

We can define ***VIROO Teleport Points*** as locations of interest in the scenery.

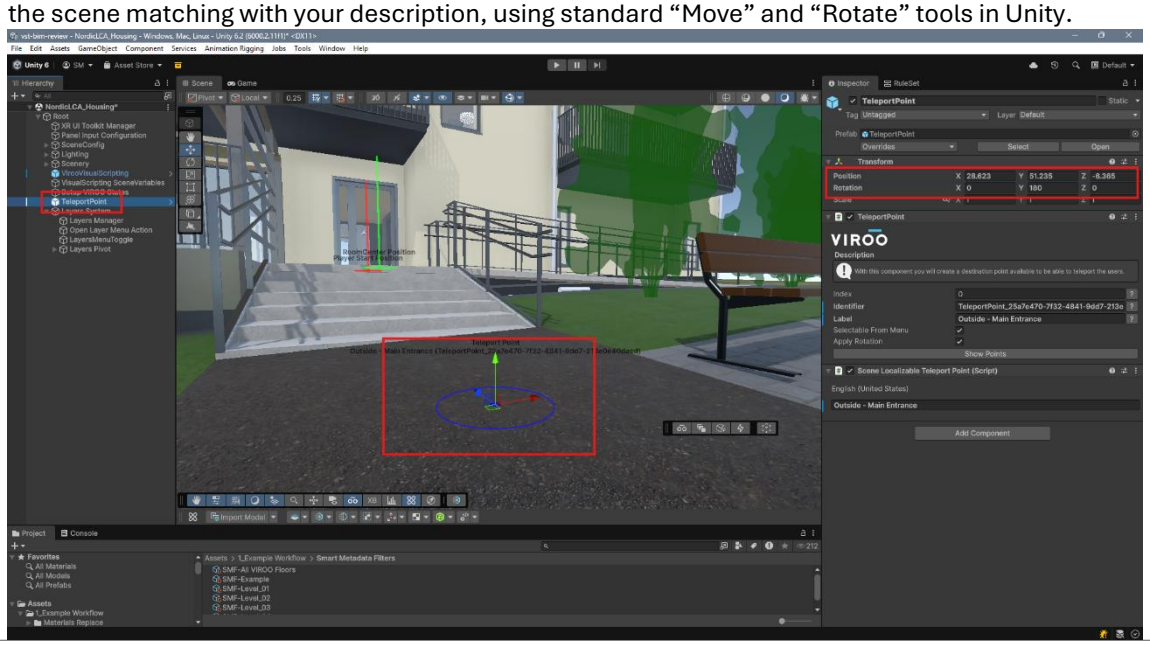
Adding Teleport Points is the way to feed the VIROO Standard Teleport Menu.

To add a teleport point, proceed as follows: Using the ***Viroo Toolbar***, just click on ***Teleport Point*** and fill in information. Fill in the Important fields which are ***Index***, representing its position on the menu and need to be different in any instance of teleport point, and ***Name*** (can be localized into several languages).



Then click on ***Accept***, and a new GameObject containing the information about the teleport point will be created.

The position and rotation of a Teleport Point rely on its ***transform***, so move it to a proper location in the scene matching with your description, using standard “Move” and “Rotate” tools in Unity.

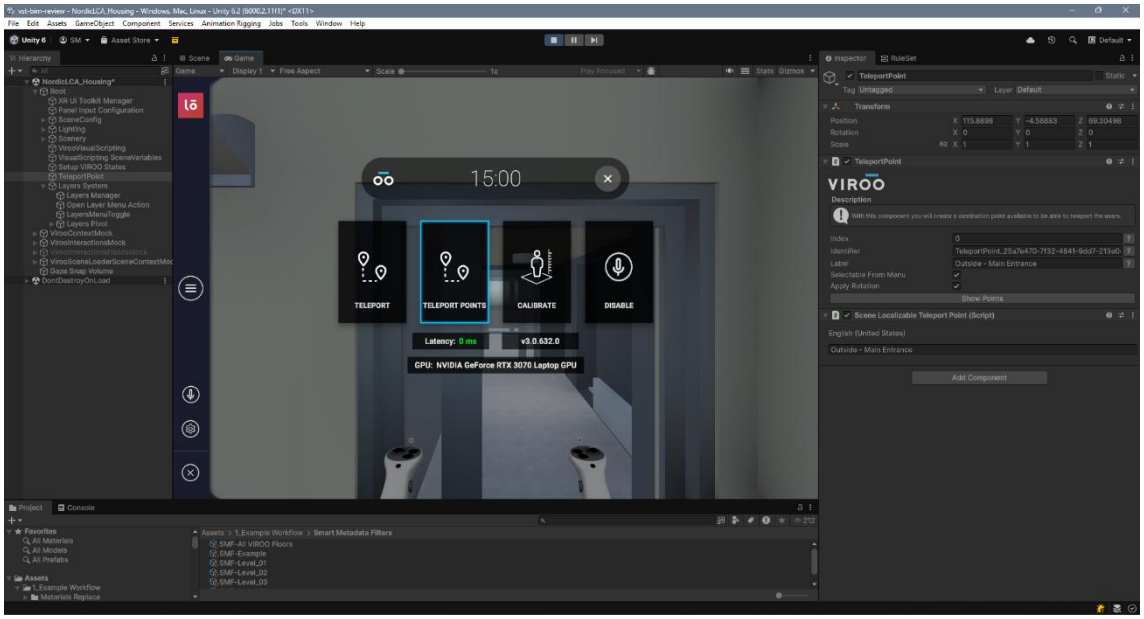


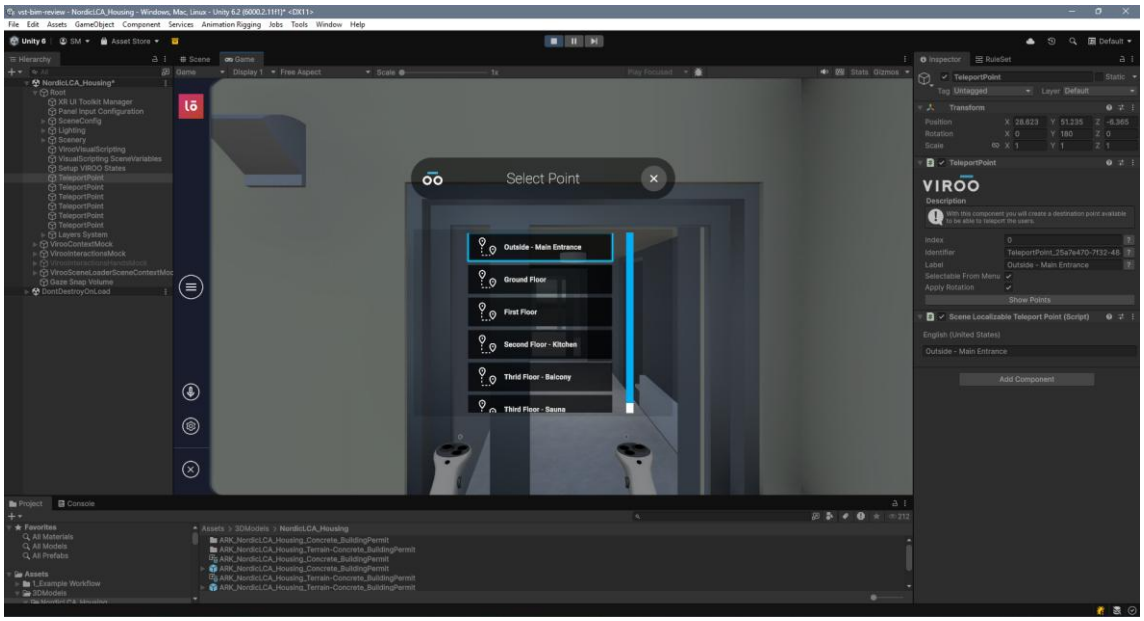
Copyright 2025 Virtualware Company - All rights reserved.

In Unity the ***forward axis*** is represented by ***"z" component "in blue" color***, so if you want it to be pointing the entrance door, rotate the transform for the ***blue axis gizmo*** to be aimed toward the door, as shown in the previous image.

USING TELEPORT POINTS IN RUNTIME

Then, when playing the scene in VIROO, a new entry will appear in the Teleport Points section of the ***VIROO Standard Menu.***



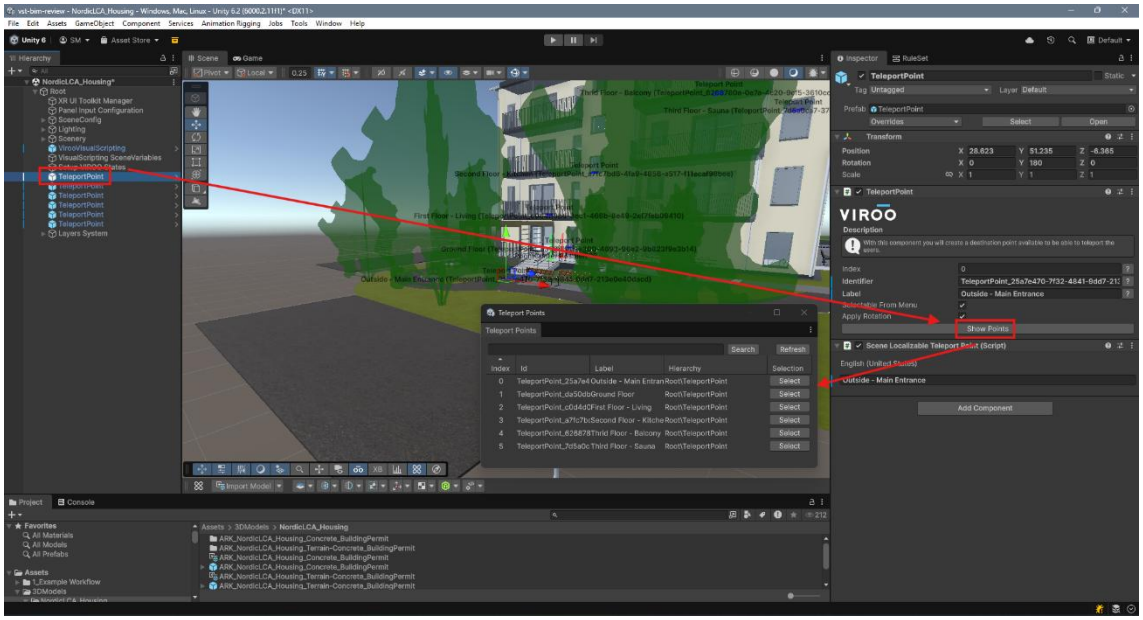


Copyright 2025 Virtualware Company - All rights reserved.

THE TELEPORT POINTS INSPECTOR

We provide a convenient panel in VIROO to keep track of the teleport points and allow them to be selected. Once selected, their properties can be modified.

To open this panel, just select one of the ***Teleport Points*** in the Hierarchy, and then click on the [***Show Points***] button in its component inspector.



Copyright 2025 Virtualware Company - All rights reserved.

VIRŌŌ